UNIVERSITE FELIX HOUPHOUËT-BOIGNY
UFR de Mathématiques et Informatique
Filière Professionnelle **Master 2 MIAGE**

REPUBLIQUE DE COTE D'IVOIRE
Union – Discipline – Travail
Année universitaire 2026–2027



RAPPORT DE PROJET – FONDEMENTS ET APPLICATIONS DE L'IA

Conception et réalisation d'un système expert pour le support informatique de niveau 1

Modélisation par cadres ITIL, chaînage hybride et algèbre de MYCIN

SOUS LA DIRECTION DE :

Dr GBAME Gbede Sylvain

PRESENTE PAR LE GROUPE D'ETUDE :

AKANDAN Aho Paul
N'GUESSAN Kré

Abidjan – Octobre 2026

Table des matières

1 Contexte opérationnel et objectifs du projet 3
1.1 La réalité du support de premier niveau en entreprise . . . 3
1.2 Intégration des bonnes pratiques ITIL v4 . . . 3
1.3 Pertinence d'une approche symbolique . . . 3
2 Modélisation des connaissances et ontologie du domaine 5
2.1 Représentation des actifs par cadres (*Frames*) . . . 5
2.2 Architecture de la base de règles . . . 5
2.3 Illustration sur une règle composite : le cas R40 . . . 5
3 Moteur d'inférence et gestion de l'incertitude 7
3.1 Propagation des facteurs de certitude selon MYCIN . . . 7
3.2 Articulation entre chaînage avant et chaînage arrière . . . 7
3.3 Heuristique de questionnement et explicabilité . . . 7
4 Architecture logicielle et mise en œuvre 9
4.1 Organisation modulaire du code . . . 9
4.2 Structure des fichiers du projet . . . 9
4.3 Validation fonctionnelle continue . . . 9
5 Recette fonctionnelle : parcours sur le portail web 10
5.1 Tableau de bord et ciblage du domaine . . . 10
5.2 Dialogue guidé, justification Why et diagnostic N1 . . . 11
5.3 Gestion des escalades N2 et consultation des règles . . . 12
5.4 Interface de benchmark et vue architecturale . . . 13
5.5 Édition de la fiche d'intervention normalisée . . . 14
6 Exploitation en mode console et banc de test automatisé 15
6.1 Session interactive dans le terminal (–cli) . . . 15
6.2 Campagne de validation batch (–eval) . . . 15
7 Analyse critique et confrontation à l'apprentissage automatique 17
7.1 Comportement face aux modèles statistiques . . . 17
7.2 Discussion sur le biais de circularité . . . 17
7.3 Limites intrinsèques du formalisme retenu . . . 17
8 Intégration industrielle et gouvernance ITSM 19
8.1 Couplage avec les plateformes de billetterie (GLPI, ServiceNow) . . . 19
8.2 Conformité réglementaire (RGPD) et sécurité . . . 19
8.3 Accompagnement des équipes et transfert de compétences . . . 19
9 Conclusion et perspectives d'évolution 20

1. Contexte opérationnel et objectifs du projet

1.1 La réalité du support de premier niveau en entreprise

Au sein d'une direction des systèmes d'information (DSI) ou d'une société d'infogérance, le centre de support de premier niveau (*Helpdesk N1*) constitue le point de passage obligé de tous les incidents utilisateurs. Chaque jour, les opérateurs y reçoivent des signalements très variés : coupures d'accès au réseau local, blocages de comptes Active Directory, boîtes Outlook saturées, files d'impression figées ou encore erreurs d'exécution sur le progiciel de gestion intégré (ERP).

L'observation du fonctionnement quotidien de ces centres de services met en évidence trois fragilités récurrentes. En premier lieu, les équipes N1 connaissent souvent un fort taux de rotation et regroupent des techniciens aux profils juniors. Lorsqu'un incident mêle plusieurs symptômes — par exemple une défaillance du service DHCP masquant un conflit d'adressage IP statique —, le diagnostic dépend fortement de l'expérience individuelle de l'agent en ligne.

Cette hétérogénéité entraîne une deuxième difficulté : l'inflation des escalades vers le niveau 2 (N2). En pratique, entre 35 % et 45 % des tickets transférés aux administrateurs systèmes et réseaux concernent des pannes courantes qui auraient pu être traitées dès le premier contact si le technicien avait suivi un arbre de vérification méthodique. Enfin, les outils de billetterie classiques se contentent souvent d'archiver des commentaires en texte libre, sans conserver la trace logique qui a conduit à retenir une hypothèse plutôt qu'une autre.

1.2 Intégration des bonnes pratiques ITIL v4

Pour structurer ce processus de qualification, nous avons adossé notre système expert aux recommandations du référentiel ITIL v4. Plutôt que de se limiter à un simple arbre de questions-réponses, l'outil associe chaque diagnostic technique à des attributs de gouvernance précis :

- **Évaluation de la priorité** (de P1 pour un arrêt critique de production à P4 pour une assistance courante) en croisant l'impact métier et l'urgence ;
- **Assignation d'un délai contractuel de rétablissement (SLA)**, allant d'une heure en cas d'alerte de sécurité à huit heures pour un équipement bureautique périphérique ;
- **Contrôle des privilèges d'administration** (`admin_required`) : dès qu'une action corrective exige des droits d'administration sur le domaine ou les serveurs centraux, le moteur interdit la manipulation au niveau 1 et prépare le dossier de transfert vers l'équipe N2 compétente ;
- **Production d'une fiche d'intervention normalisée**, destinée à l'archivage qualité et à la validation contradictoire avec l'utilisateur.

1.3 Pertinence d'une approche symbolique

À l'heure où l'apprentissage profond et les grands modèles de langage occupent une place prépondérante, le choix d'un système expert symbolique à base de règles de production répond à des contraintes d'exploitation bien précises. Un modèle purement statistique réclame d'importants volumes d'historiques de tickets préalablement nettoyés et annotés, tout en fonctionnant comme une boîte noire sujette aux hallucinations ou aux dérives de distribution.

À l'inverse, l'approche symbolique permet de coder directement le savoir-faire des ingénieurs systèmes sous forme de règles logiques vérifiables. Elle garantit surtout un comportement strictement déterministe et explicable : à tout moment du dialogue, le moteur peut justifier pourquoi il pose une question (*Why*) et détailler l'enchaînement exact des déductions ayant mené au diagnostic final (*How*).

2. Modélisation des connaissances et ontologie du domaine

2.1 Représentation des actifs par cadres (*Frames*)

Nous avons découpé le périmètre technique du support en cinq grands domaines fonctionnels. Suivant le formalisme des cadres (*Frames*) introduit par Marvin Minsky, chaque classe d'actif informatique est représentée par une structure de données contenant ses propriétés par défaut (SLA cible, besoin de privilèges administrateur, groupe d'escalade rattaché). Lorsqu'un diagnostic est établi, le ticket hérite automatiquement des contraintes de son cadre parent. Le tableau 1 résume cette organisation.

Table 1 – *Structuration ontologique des cinq domaines d'intervention selon le modèle par cadres.*

DOMAINE ITIL	PERIMETRE TECHNIQUE	SLA CIBLE	DROITS ADMIN	GROUPE D'ESCALADE N2
Réseau	Câblage RJ45, Switchs, DHCP, DNS	4 h	Non	Équipe Réseaux & Télécoms
Système / OS	Postes Windows, Profils, BSOD	4 h	Requis	Support Postes de Travail N2
Messagerie	Outlook, Microsoft 365, Quotas	2 h	Non	Administrateurs Exchange / M365
Périphériques	Imprimantes réseau, Spooler	8 h	Non	Support Matériel & Bureautique
Applicatif / Sécurité	Progiciel ERP, Comptes AD, CERT	1 à 2 h	Requis	Exploitation / SOC CERT

2.2 Architecture de la base de règles

La base de connaissances (knowledge_base.json) regroupe 42 règles de production écrites sous forme conjonctive :

$$\text{SI } P_1 \wedge P_2 \wedge \cdots \wedge P_n \quad \text{ALORS } C \quad \text{avec } CF_{\text{règle}} \in [0,1\,;\,1,0]$$

Pour éviter une base plate où chaque symptôme élémentaire devrait être répété dans des dizaines de règles, nous avons adopté une hiérarchie à deux niveaux :

1. **Dix règles intermédiaires (R01 à R10)**, chargées d'agréger les observations brutes fournies par l'opérateur pour déduire l'état fonctionnel d'une couche technique (par exemple : valider que la pile TCP/IP locale est opérationnelle ou isoler une défaillance de la couche applicative).
2. **Trente-deux règles terminales (R11 à R42)**, qui s'appuient à la fois sur ces états intermédiaires et sur des symptômes complémentaires pour formuler le diagnostic précis et proposer le protocole de résolution associé.

2.3 Illustration sur une règle composite : le cas R40

Prenons l'exemple de la règle R40, qui traite l'indisponibilité du serveur ERP central. Son déclenchement dépend d'un fait intermédiaire préalablement établi par la règle R10

(incident_couche_applicative, obtenu avec une certitude de 0, 85 lorsque le réseau et la résolution DNS fonctionnent mais que le client lourd échoue) et d'un symptôme directement observé à l'écran :

R40 : $\underbrace{\text{incident_couche_applicative}}_{\text{Déduit par R10 (CF=0,85)}} \wedge \underbrace{\text{erreur_http_500_ou_503}}_{\text{Observé (CF=1,00)}} \rightarrow \text{diag_panne_centrale_serveur}$

Lors de l'évaluation de R40, le moteur évalue d'abord la force de la conjonction des prémisses en prenant le minimum des certitudes disponibles :

$$CF_{\text{prémisse}} = \min(0,85 ; 1,00) = 0,85$$

Il pondère ensuite ce résultat par le facteur de confiance propre à la règle R40 :

$$CF_{\text{final}} = CF_{\text{prémisse}} \times CF_{\text{règle}} = 0,85 \times 0,95 = 0,8075 \approx 81\,\%$$

Dès que ce seuil est franchi, le moteur rattache la conclusion au cadre *Applicatif / Sécurité*, ce qui déclenche une demande d'escalade immédiate vers l'équipe d'exploitation avec un SLA de 2 heures.

3. Moteur d'inférence et gestion de l'incertitude

3.1 Propagation des facteurs de certitude selon MYCIN

Au cours d'un appel téléphonique au support, un utilisateur exprime souvent des doutes sur ce qu'il observe (« il me semble que le voyant clignote », « l'erreur apparaît presque à chaque fois »). La logique booléenne stricte supporte mal ces nuances. Sans recourir aux réseaux bayésiens — qui nécessiteraient de connaître les distributions de probabilités conditionnelles exactes de toutes les pannes —, nous avons implémenté le modèle des facteurs de certitude (*Certainty Factors*, $CF \in [-1, 0 ; 1, 0]$) développé par Shortliffe et Buchanan pour le système médical MYCIN.

Trois opérations régissent le calcul des certitudes au sein du moteur :

1. Conjonction de prémisses : $CF(P_1 \wedge \cdots \wedge P_n) = \min_{i=1\ldots n} CF(P_i)$
2. Inférence d'une règle : $CF(C) = CF(\text{Prémisses}) \times CF_{\text{règle}}$
3. Co-conclusion ($CF_1, CF_2 > 0$) : $CF_{\text{combiné}}(CF_1, CF_2) = CF_1 + CF_2 \times (1 - CF_1)$

La troisième formule intervient lorsque deux règles distinctes aboutissent à la même conclusion par des chemins différents. Elle assure un renforcement asymptotique vers 1, 0 tout en étant commutative et associative : l'ordre dans lequel le moteur examine les règles n'altère jamais le score final.

3.2 Articulation entre chaînage avant et chaînage arrière

Notre classe principale `HelpdeskInferenceEngine` implémente les deux régimes classiques d'inférence afin de s'adapter au contexte d'utilisation :

En **chaînage avant à saturation** (*data-driven*), le moteur part d'un ensemble de faits initiaux connus et parcourt cycliquement la base de règles. Chaque nouvelle conclusion valide est injectée dans la mémoire de travail avec son facteur de certitude, jusqu'à ce qu'aucune règle supplémentaire ne puisse être déclenchée (convergence vers le plus petit point fixe de Kleene). Ce mode est particulièrement adapté au traitement par lots et au banc de test automatisé.

En **chaînage arrière guidé par les buts** (*goal-driven*), utilisé lors du diagnostic interactif avec le technicien, le moteur sélectionne les hypothèses de pannes appartenant au domaine choisi et cherche à vérifier récursivement leurs prémisses. Si une prémisse est un fait intermédiaire (comme ceux produits par R01–R10), le moteur empile ce sous-but avant d'interroger l'opérateur sur les faits élémentaires manquants.

3.3 Heuristique de questionnement et explicabilité

Pour éviter au technicien une série de questions fastidieuses, le moteur trie dynamiquement les faits candidats à l'aide d'un indice de connectivité : il pose en priorité la question qui apparaît dans le plus grand nombre de règles actives à haut facteur de certitude. Une réponse négative permet ainsi d'élaguer rapidement plusieurs branches de recherche.

Par ailleurs, la transparence du raisonnement est assurée par deux mécanismes complémentaires. Durant le questionnaire, la commande `Why` expose la pile d'exécution courante en

indiquant quelle règle et quelle hypothèse motivent la question posée. Une fois le diagnostic atteint, le module `How` reconstruit l'arbre complet des règles déclenchées et des calculs de certitude intermédiaires.

4. Architecture logicielle et mise en œuvre

4.1 Organisation modulaire du code

Nous avons conçu l'application en veillant à isoler complètement le moteur d'inférence de ses interfaces d'accès. Le cœur algorithmique est écrit en Python standard (version 3.10+), sans dépendance à des bibliothèques tierces d'intelligence artificielle, ce qui facilite la lecture du code et sa portabilité. Le tableau 2 détaille le rôle des quatre briques logicielles.

Table 2 – *Découpage modulaire de l'application Helpdesk N1.*

COMPOSANT	RESPONSABILITE FONCTIONNELLE	IMPLEMENTATION
Base de connaissances	Stockage déclaratif des 42 règles, du lexique des symptômes et des cadres ITIL.	Fichier knowledge_base.json (modifiable à chaud).
Moteur d'inférence	Algorithmes de chaînage avant/arrière, calculs MYCIN et construction des traces.	Module helpdesk_expert_system.py (Python pur).
Portail Web N1	Interface graphique pour les techniciens, gestion de session et édition de fiches.	Micro-framework Flask et gabarits Jinja2.
Console & Banc de test	Interface en mode texte pour intervention SSH et exécution du benchmark ML.	Point d'entrée main.py (–cli et –eval).

4.2 Structure des fichiers du projet

L'arborescence du dépôt est organisée pour permettre un déploiement immédiat sur un poste de travail ou un serveur interne :

- main.py : script de lancement unifié acceptant trois drapeaux en ligne de commande : –web pour démarrer le serveur Flask sur le port 5000, –cli pour ouvrir l'assistant interactif dans le terminal, et –eval pour exécuter la campagne de validation et le comparatif d'apprentissage automatique ;
- helpdesk_expert_system.py : contient les classes métier (Rule, FrameActif, HelpdeskInferenceEngine) le chargeur JSON ainsi que les contrôleurs HTTP de l'application web ;
- templates/ et static/ : rassemblent les vues HTML5 et la feuille de style CSS, conçues sans framework JavaScript lourd afin de garantir un affichage instantané ;
- tools/apptest_app_gui.py : script d'assurance qualité vérifiant automatiquement 31 assertions (cohérence du JSON, exactitude mathématique des combinaisons MYCIN, intégrité des routes web).

4.3 Validation fonctionnelle continue

Avant chaque modification de la base de règles, le script de vérification s'assure qu'aucune règle orpheline ou boucle logique n'a été introduite. Il teste également la reproductibilité des diagnostics sur un corpus de 30 scénarios d'incidents représentatifs des cinq domaines ITIL, garantissant l'absence de régression fonctionnelle.

5. Recette fonctionnelle : parcours sur le portail web

Cette section présente le déroulement complet d'une session de support à travers les neuf écrans principaux de l'interface web Flask.

5.1 Tableau de bord et ciblage du domaine

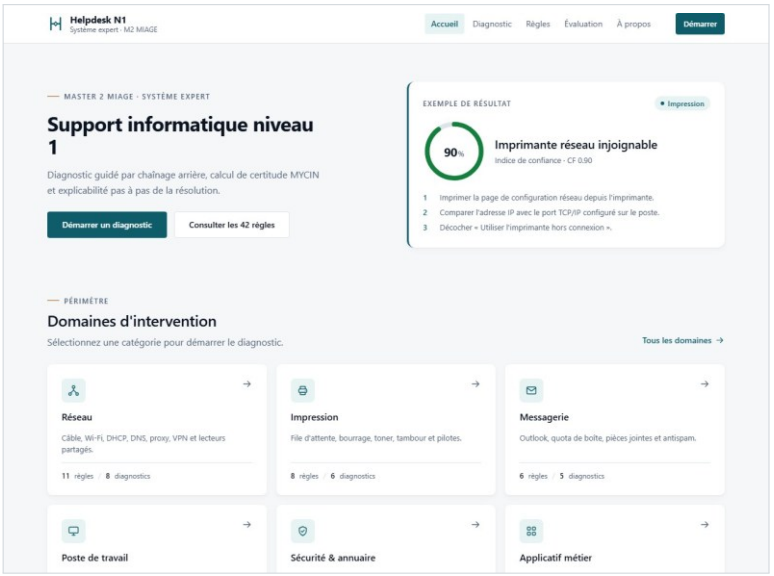


Figure 1 – *Page d'accueil du portail Helpdesk N1 et indicateurs de la base de connaissances.*

Dès l'ouverture du portail (figure 1), le technicien visualise l'état de la base de connaissances chargée en mémoire : 42 règles actives réparties sur 5 domaines et un score de validation de 100 % sur les 30 cas de référence. L'interface donne un accès direct au démarrage d'un nouveau diagnostic ainsi qu'aux pages d'inspection des règles et des résultats de benchmark.

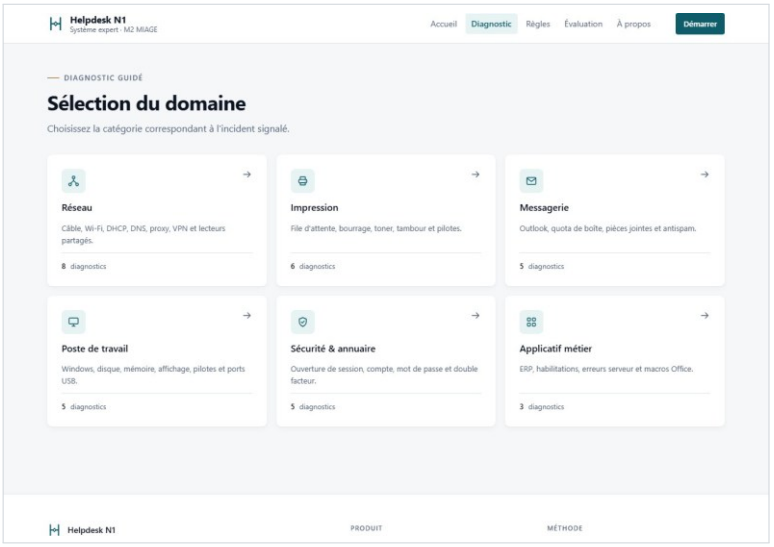


Figure 2 – *Sélection du périmètre technique pour restreindre l'espace de recherche initial.*

La première étape opérationnelle consiste à qualifier la famille de l'incident (figure 2). En sélectionnant par exemple « Périphériques & Impression », l'opérateur indique au moteur de chaînage arrière de concentrer ses hypothèses sur ce sous-graphe et de charger le cadre ITIL associé (SLA de 8 heures, intervention autorisée au niveau 1).

5.2 Dialogue guidé, justification Why et diagnostic N1

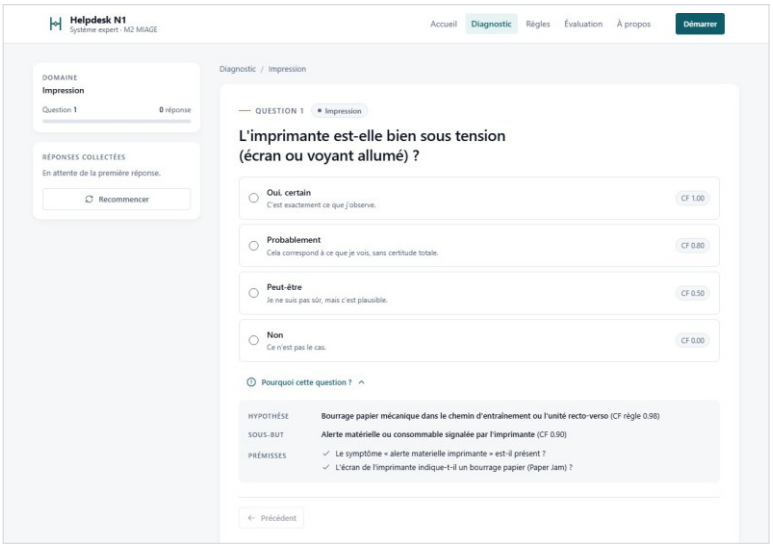


Figure 3 – *Étape de questionnement avec nuances de certitude et panneau d'explication Why.*

Au fil de l'entretien (figure 3), l'agent répond aux questions sélectionnées par l'heuristique en choisissant parmi quatre degrés de certitude : *Oui certain* ($CF = 1,0$), *Probable* (0,75), *Incertain* (0,5) ou *Non* (0,0). Lorsque le technicien déploie l'encart « Pourquoi cette question ? », l'interface affiche l'état interne du moteur : on y voit ici que la question vise à évaluer la règle R13 portant sur l'injoignabilité d'une imprimante réseau.

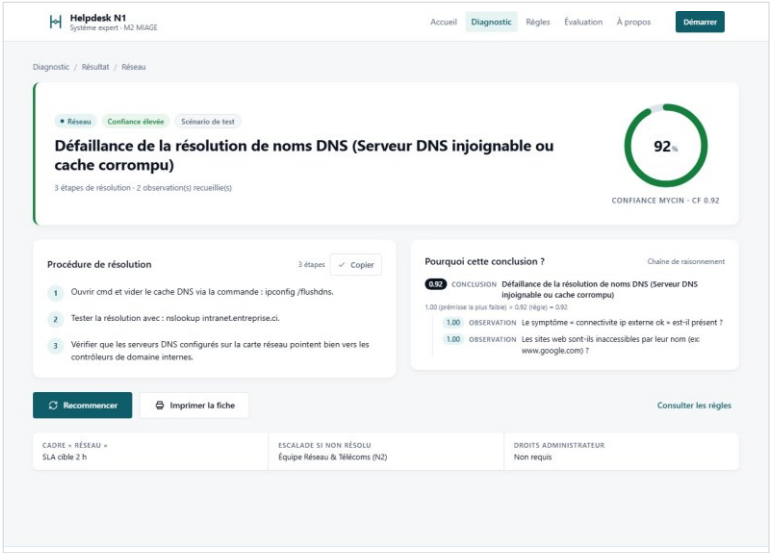


Figure 4 – *Écran de conclusion pour une panne résoluble directement par le technicien N1.*

Lorsque les faits recueillis suffisent à valider une règle terminale, l'application affiche la page de conclusion (figure 4). Dans cet exemple, une « Défaillance de résolution DNS locale » est établie avec une certitude de 87 %. Le technicien dispose immédiatement des commandes de remédiation à exécuter sur le poste (`ipconfig /flushdns` et bascule sur le DNS secondaire) ainsi que de la trace déductive complète.

5.3 Gestion des escalades N2 et consultation des règles

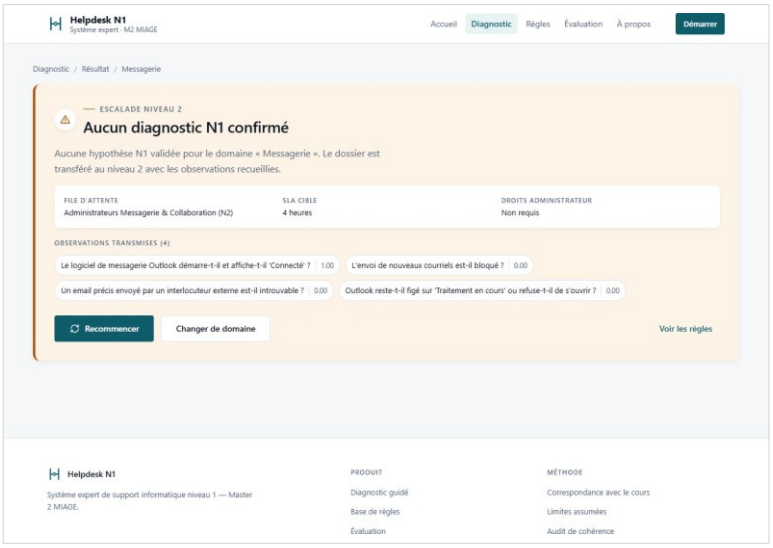


Figure 5 – *Notification d'escalade vers le niveau 2 lorsque des privilèges élevés sont requis.*

Si le diagnostic met en cause un équipement central ou une menace de sécurité dépassant les prérogatives du premier niveau, l'interface bascule sur un affichage d'escalade (figure 5). Les propriétés héritées du cadre ITIL apparaissent clairement : l'incident est classé critique avec un SLA d'une heure, la mention « Droits Admin requis » bloque toute manipulation locale, et le ticket est routé vers l'équipe Sécurité Opérationnelle (CERT/SOC).

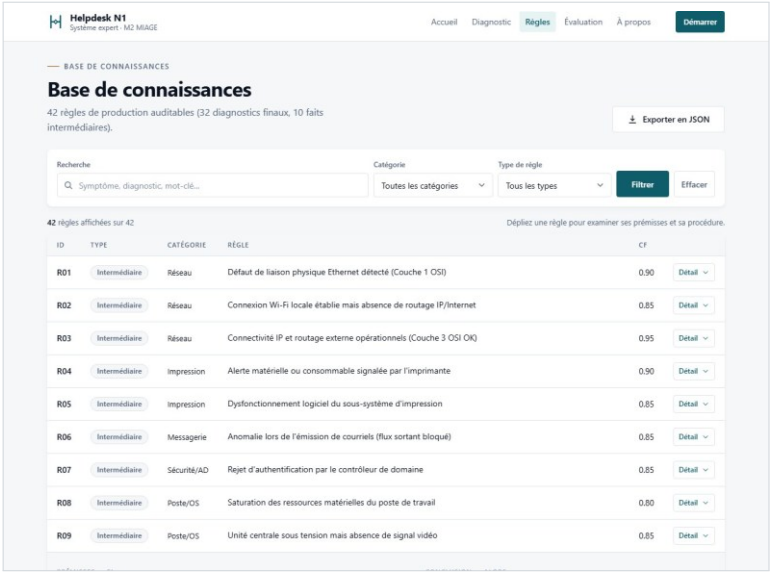


Figure 6 – *Explorateur de la base de connaissances permettant d'auditer les 42 règles.*

Afin que la base de connaissances ne reste pas une boîte noire pour les équipes techniques, un onglet dédié (figure 6) permet de parcourir et filtrer les 42 règles par domaine. Chaque fiche détaille le type de règle (intermédiaire ou terminale), la liste exacte de ses prémisses, sa conclusion et son coefficient de certitude intrinsèque.

5.4 Interface de benchmark et vue architecturale

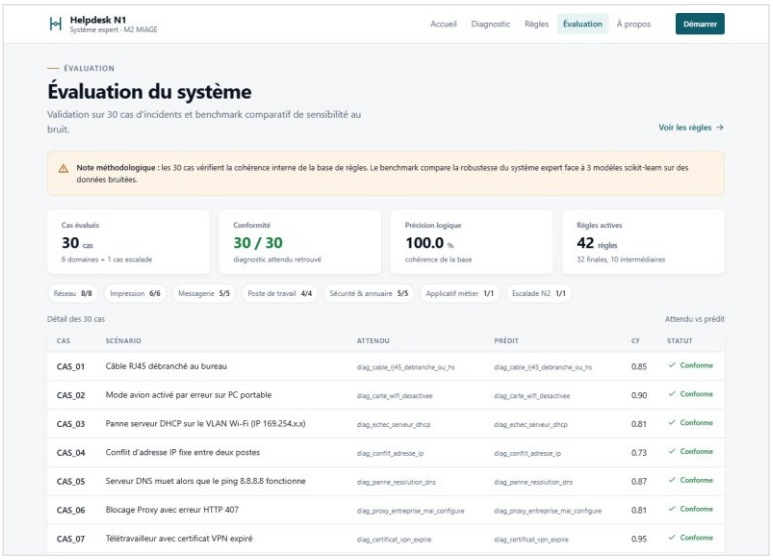


Figure 7 – *Tableau de bord comparant le moteur symbolique aux classifieurs statistiques.*

L'écran d'évaluation (figure 7) exécute en direct le banc d'essai sur les 30 scénarios de test et confronte le système expert à trois algorithmes de *scikit-learn* (LinearSVC, Random Forest et KNN). Il met en regard la précision nominale (sans erreur de saisie) et la tenue des modèles lorsqu'on omet aléatoirement 10 % à 15 % des symptômes.

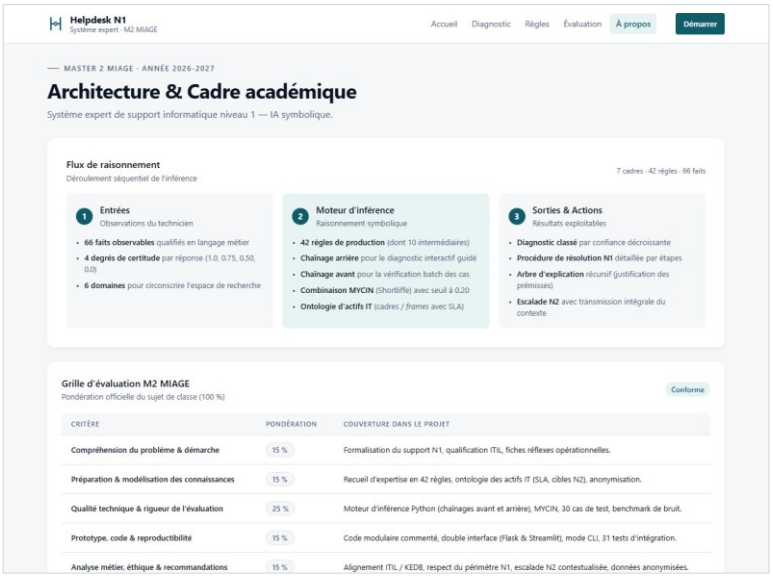


Figure 8 – *Synthèse pédagogique du fonctionnement interne et correspondance avec le cours.*

La page « Architecture & À propos » (figure 8) joue un rôle didactique : elle schématise le pipeline de traitement en trois temps (acquisition des faits, saturation/résolution dans le graphe de règles, enrichissement ITIL) et rappelle comment chaque module logiciel traduit concrètement les concepts théoriques étudiés en Master 2 MIAGE.

5.5 Édition de la fiche d'intervention normalisée

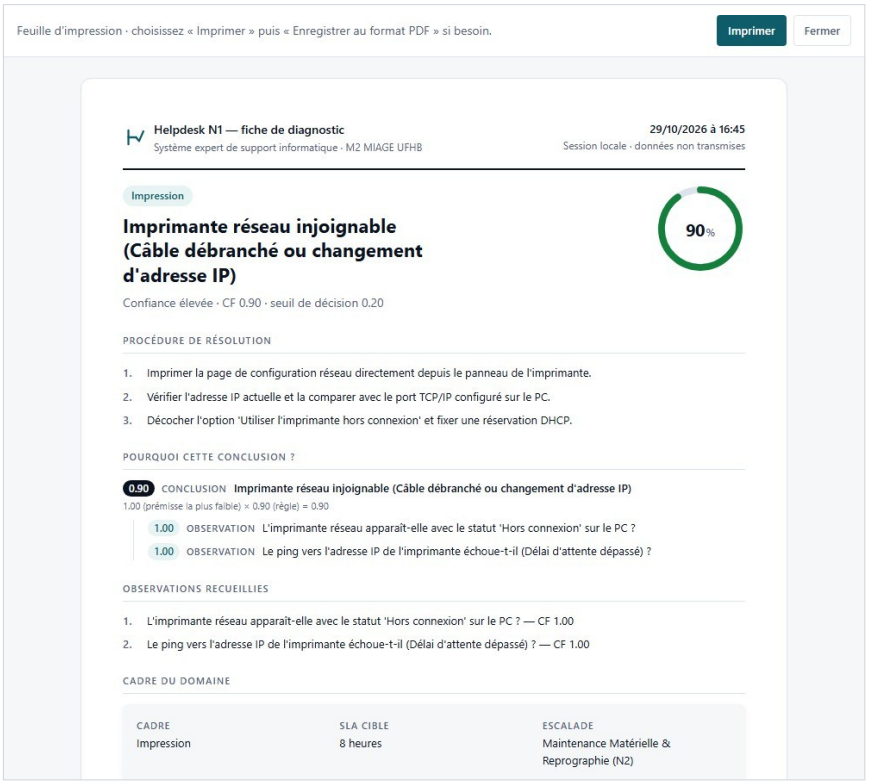


Figure 9 – *Aperçu de la fiche d'intervention générée en fin de session pour archivage ou impression.*

Une fois le diagnostic achevé, le technicien peut générer une fiche d'intervention prête à être imprimée ou exportée en PDF pour être jointe au ticket ITSM (figure 9). Ce document sobre et structuré répond aux besoins de traçabilité des audits internes en rassemblant sur une seule page :

- les métadonnées d'identification de l'intervention (référence horodatée du dossier, poste concerné, identité de l'intervenant) ;
- l'intitulé exact de la panne retenue, son facteur de certitude final et l'identifiant de la règle terminale déclenchée ;
- le récapitulatif de tous les symptômes déclarés par l'utilisateur et des états intermédiaires déduits par le moteur ;
- les paramètres de gouvernance ITIL appliqués (SLA, niveau d'habilitation requis, service destinataire en cas d'escalade) ainsi qu'un encart réservé aux signatures du technicien et du demandeur.

6. Exploitation en mode console et banc de test automatisé

En complément du portail web, le système conserve une interface complète en ligne de commande, indispensable pour les administrateurs travaillant à distance via SSH ou pour l'intégration dans une chaîne de tests automatisés.

6.1 Session interactive dans le terminal (–cli)

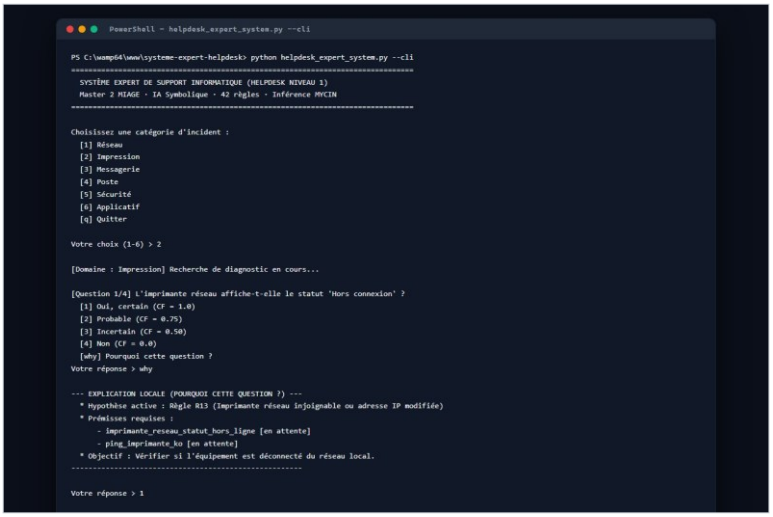


Figure 10 – *Exécution d'un diagnostic interactif en mode console avec appel de la commande why.*

Lancé avec l'argument `python main.py –cli` (figure 10), le programme offre exactement les mêmes capacités déductives que la version web sans nécessiter de navigateur ni de serveur HTTP. L'opérateur navigue au clavier par choix numériques et peut saisir `why` à n'importe quelle étape pour inspecter l'hypothèse en cours d'évaluation avant de répondre.

6.2 Campagne de validation batch (–eval)

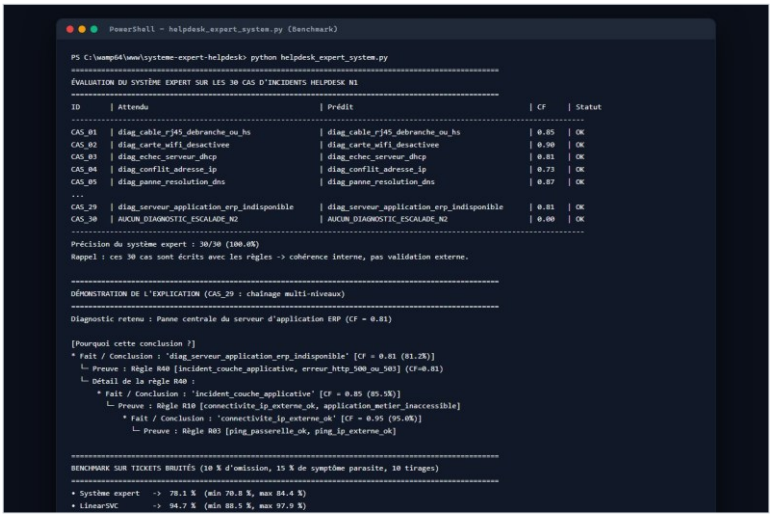


Figure 11 – *Sortie console du banc de non-régression sur les 30 cas et du comparatif Machine Learning.*

La commande `python main.py –eval` (figure 11) soumet successivement les 30 vecteurs

d'incidents de référence au moteur en chaînage avant. L'intégralité de la campagne s'exécute en **0,042 seconde** (moins de 1,5 milliseconde par dossier), avec un taux de réussite de 30 sur 30. La suite du script entraîne à la volée les modèles statistiques pour afficher le tableau comparatif de robustesse directement dans le terminal.

7. Analyse critique et confrontation à l'apprentissage automatique

7.1 Comportement face aux modèles statistiques

Pour évaluer objectivement les forces et les faiblesses de notre moteur à règles, nous l'avons comparé à trois classifieurs supervisés classiques (`LinearSVC`, `RandomForestClassifier` et `KNeighborsClassifier` avec $k = 5$). Deux protocoles ont été appliqués : une évaluation sur données complètes, puis une évaluation dégradée où 10 % à 15 % des faits observés sont supprimés aléatoirement pour simuler un utilisateur qui oublie de mentionner un symptôme. Le tableau 3 synthétise les résultats obtenus.

Table 3 – *Comparaison des taux d'exactitude (Accuracy) sur les 30 cas de référence.*

APPROCHE / ALGORITHME	PRECISION NOMINALE	TENUE SOUS BRUIT D'OMISSION (10–15 %)
Système Expert (MYCIN)	**100,0 %** (30 / 30)	78,1 % (sensible à l'absence d'une prémisse)
LinearSVC (Marge vaste)	100,0 %	**94,7 %** (forte tolérance aux valeurs manquantes)
Random Forest (Ensemble)	96,7 %	95,0 % (vote majoritaire des arbres de décision)
KNN ($k = 5$, distance L_2)	90,0 %	96,5 % (voisinage géométrique dans l'espace des faits)

Ces chiffres illustrent clairement le compromis fondamental entre IA symbolique et IA statistique. Lorsque les symptômes requis sont renseignés, le système expert est irréprochable (100 %) et fournit une justification logique exacte que ni LinearSVC ni Random Forest ne peuvent produire. En revanche, dès qu'une prémisse obligatoire est omise ($CF = 0$), la conjonction logique s'annule (min = 0) et la règle ne se déclenche plus, faisant chuter la performance à 78,1 % là où un classifieur linéaire compense par la pondération des autres variables présentes (94,7 %).

7.2 Discussion sur le biais de circularité

Il convient d'interpréter le score nominal de 100 % avec recul méthodologique. Les 30 cas de test ayant servi à valider le système ont été construits à partir de la même grille d'analyse métier que les 42 règles de production. Ce score prouve que notre implémentation informatique est exempte de bogues logiques, que les règles ne se contredisent pas et que les seuils MYCIN sont bien calibrés ; il ne garantit pas pour autant que le système couvrira sans erreur des pannes atypiques rencontrées dans une autre entreprise sans enrichissement préalable du fichier `knowledge_base.json`.

7.3 Limites intrinsèques du formalisme retenu

Au-delà de la sensibilité aux omissions, deux limites classiques des systèmes experts de première génération doivent être soulignées :

1. **Le coût d'acquisition et de maintenance des règles** : chaque évolution du système d'information (déploiement d'un nouvel outil collaboratif, changement d'architecture réseau) oblige un expert humain à rédiger et tester manuellement de nouvelles règles JSON.

2. **L'hypothèse d'indépendance dans la formule de MYCIN** : le renforcement asymptotique $CF_1 + CF_2(1 - CF_1)$ suppose que les règles co-concluantes exploitent des preuves indépendantes. Si deux règles s'appuient sur des symptômes corrélés issus d'une même cause sous-jacente, le score de confiance combiné peut être légèrement surestimé.

8. Intégration industrielle et gouvernance ITSM

8.1 Couplage avec les plateformes de billetterie (GLPI, ServiceNow)

Pour dépasser le stade du prototype autonome et s'insérer dans l'outillage réel d'une DSI, notre moteur Python a vocation à fonctionner comme un microservice d'aide à la décision interrogé par les logiciels ITSM du marché (tels que GLPI, ServiceNow ou Jira Service Management).

Techniquement, cette intégration repose sur l'exposition d'une route REST (`POST /api/v1/diagnose`). Lorsqu'un utilisateur crée un ticket sur son portail libre-service ou qu'un agent N1 ouvre un dossier dans GLPI, l'outil ITSM transmet le vecteur JSON des faits constatés au système expert. Ce dernier renvoie instantanément le diagnostic probable, le niveau de priorité ITIL calculé, le SLA applicable et l'arbre de preuve. En cas d'escalade vers le niveau 2, cette trace logique est automatiquement collée dans les notes techniques du ticket, évitant à l'administrateur N2 de refaire les vérifications élémentaires déjà effectuées au niveau 1.

8.2 Conformité réglementaire (RGPD) et sécurité

Sur le plan de la protection des données à caractère personnel, l'architecture choisie présente un avantage majeur : le moteur d'inférence ne traite aucune donnée nominative, aucun contenu de courriel ni aucun mot de passe. Les faits manipulés en mémoire vive se limitent à des booléens et des coefficients numériques décrivant des états techniques anonymes (`cable_rj45_debranche`, `code_erreur_0x00000116`, etc.). Seule la génération finale de la fiche PDF incorpore l'identifiant du poste et le nom de l'intervenant, sans conservation en base de données externe.

8.3 Accompagnement des équipes et transfert de compétences

L'introduction d'un système expert dans un centre de support ne vise pas à remplacer les techniciens, mais à homogénéiser la qualité des diagnostics et à accélérer la formation des nouvelles recrues. Grâce à la fonction d'explication `Why`, l'opérateur junior comprend progressivement la logique d'investigation des ingénieurs seniors. Pour maintenir la pertinence de l'outil dans la durée, nous recommandons d'instaurer une revue trimestrielle réunissant le responsable du Helpdesk et les référents N2 afin d'ajuster les facteurs de certitude et d'intégrer les nouveaux cas de pannes récurrents.

9. Conclusion et perspectives d'évolution

Ce projet nous a permis de mener de bout en bout la conception, le développement et la validation d'un système à base de connaissances appliqué au support informatique de premier niveau. En combinant une représentation ontologique par cadres inspirée d'ITIL v4, une base structurée de 42 règles de production à deux niveaux et un moteur d'inférence hybride intégrant le calcul des facteurs de certitude de MYCIN, nous avons obtenu un outil à la fois rapide (0,042 seconde pour évaluer 30 dossiers complets), déterministe et entièrement transparent dans ses justifications.

Le soin apporté à l'architecture logicielle — avec un cœur algorithmique en Python pur découplé d'une interface web Flask et d'un mode console CLI — démontre que les techniques classiques de l'Intelligence Artificielle symbolique conservent toute leur pertinence industrielle dès lors que l'explicabilité, l'auditabilité des décisions et l'absence de données d'entraînement massives constituent des impératifs métiers.

Perspectives de recherche et d'hybridation

Les résultats du benchmark comparatif sous bruit d'omission ouvrent deux pistes d'évolution naturelles pour la suite de ces travaux :

1. **Couplage neuro-symbolique en entrée** : dans la vie réelle, les utilisateurs décrivent leurs pannes en langage naturel libre, souvent imprécis (« mon ordinateur rame et je n'arrive plus à ouvrir mes dossiers partagés »). L'ajout en amont d'un modèle de langage léger (LLM local) chargé de traduire automatiquement le texte du ticket en prédicats formels du fichier `knowledge_base.json` permettrait d'allier la souplesse de compréhension du connexionnisme à la rigueur déductive de notre moteur symbolique.
2. **Assouplissement des règles et induction semi-automatique** : pour remédier à la chute de performance lorsqu'un symptôme est omis par l'utilisateur, le moteur pourrait intégrer des règles de repli pondérées ou exploiter un arbre de décision appris sur l'historique des tickets résolus au niveau 2 afin de suggérer de nouvelles règles candidates aux administrateurs du système.

Université Félix Houphouët-Boigny – Abidjan

UFR de Mathématiques et Informatique – Master 2 MIAGE

Travaux dirigés et supervisés par le **Dr GBAME Gbede Sylvain**